

实验 2

AI 全栈开发 (Spec Kit)

殷悦扬

2023级管院 统计学

录制：线下课；线上课



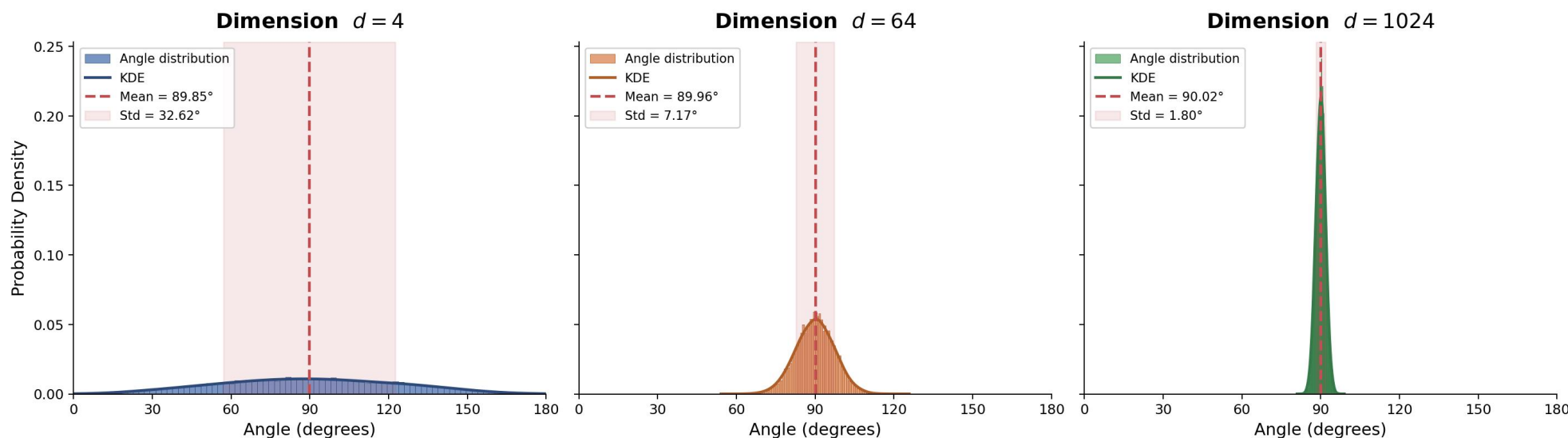
复旦大学 管理学院
SCHOOL OF MANAGEMENT
FUDAN UNIVERSITY

开智求真 拓新领变

Enlightening for Truth Pioneering for Change

- 传统位置编码是将表征位置信息的三角函数直接与 Embedding 相加，这种操作会干扰原始 Embedding 的信息吗？
 - **测度集中现象**：高维空间中随机向量的内积高度集中在 0 附近（ $\mu = 0, \sigma = \frac{1}{\sqrt{d}}$ ），即高维空间中两个独立的向量大概率正交；神经网络正是利用了高维空间的**近似正交性**，将远多于维度数量的特征以叠加的方式编码进同一表征中；但位置编码和 Embedding **不是随机向量**
 - 模型一定程度上能够容纳得下叠加的信息，但干扰仍不可忽略，后续也提出了 RoPE 等多种改进

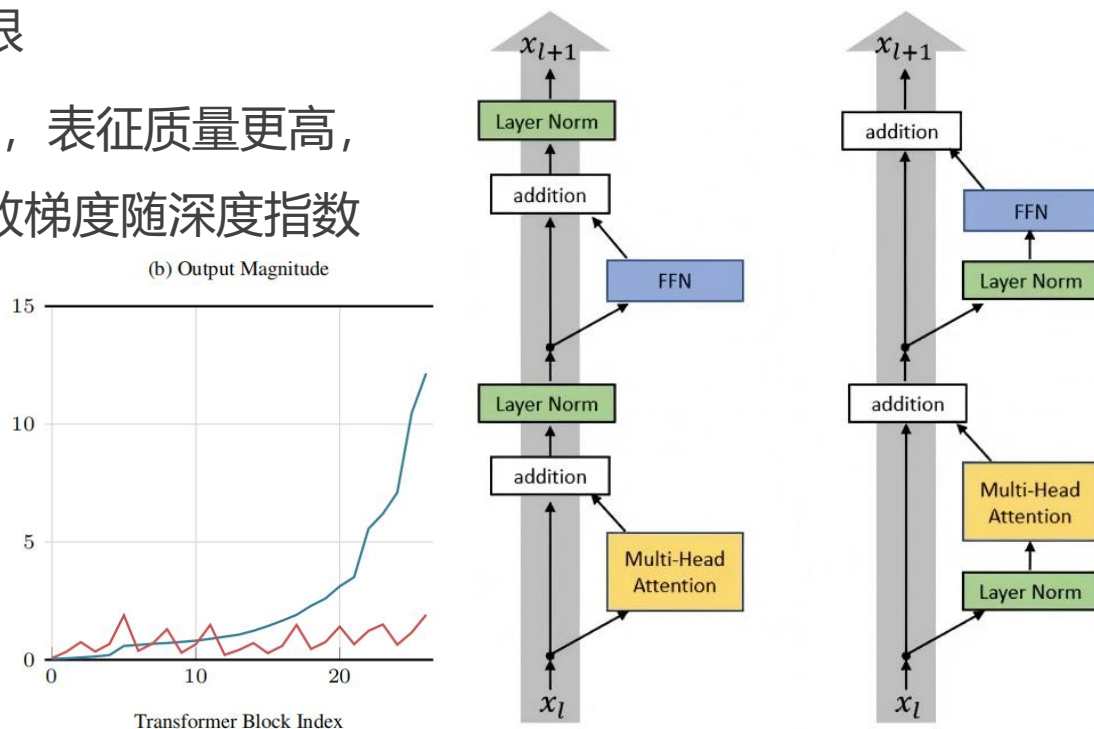
Distribution of Angles Between Random Vector Pairs



- 为什么 RMSNorm 逐渐取代 LayerNorm? Pre-Norm 和 Post-Norm 各自的优劣是什么?
 - RMSNorm**: 研究表明 LayerNorm 中的均值归一化作用有限, RMSNorm 仅保留了方差归一化, 在不损失性能的前提下, 显著降低了计算开销
 - Pre-Norm (右)**: 梯度经恒等映射直达任意层, 训练时更加稳定; 但存在 **PreNorm Dilution** 现象, 隐藏状态幅值 (如 L2 范数) 随深度呈 $O(L)$ 线性增长, 导致深层网络必须通过极大的输出才能产生影响, 早期信息被逐渐淹没, 有效深度受限
 - Post-Norm (左)**: 各层输出经归一化后幅值一致, 表征质量更高, 性能通常更好; 但归一化操作位于残差路径上, 导致梯度随深度指数衰减, 难以训练极深的网络
 - 业界权衡**: 选 Pre-Norm (训练稳定 vs. 表征质量)
 - 如何解决 Pre-Norm 的稀释问题? 核心是解决标准 **残差连接** 在深度维度上信息聚合方式的根本缺陷

Attention Residuals!

Team, Kimi, et al. "Attention Residuals."
arXiv preprint arXiv:2603.15031 (2026).





- **主要内容:**

- 安装 AI IDE / CLI / AI 插件 (如 Claude Code、Cursor、Trae 等)
- 了解 Spec Coding 与 Vibe Coding 的区别、安装 GitHub Spec Kit
- 了解 Insforge (BaaS) 平台的使用方式并配置 MCP / Skill
- 掌握如何基于 Spec Kit 与 Insforge 快速且高质量地完成全栈开发

- **课后作业:**

- 基于 Next.js 前端框架与 Insforge 后端, 使用实验 1 得到的数据, 利用 Spec Coding 开发一个全栈 Web 应用
- 选做: 探索 1-2 个好用的 Skills (如前端优化 Skill), 辅助项目开发
- 思考题 2

- **什么是 Vibe Coding?**

- 由 Andrej Karpathy 在 2025 年 2 月提出
- "完全沉浸在氛围中，拥抱指数级增长，忘记代码的存在" —— 开发者用自然语言描述需求，AI 直接生成代码
- 核心特点：对话式、即时生成、快速迭代

- **Vibe Coding 的局限性**

- 缺乏规划：一次性 prompt 生成代码，缺少系统性的需求分析和架构设计
- 质量不可控：生成结果高度依赖 prompt 质量，难以保证一致性和可维护性
- 不适合复杂项目：对于多模块、多人协作的企业级应用，Vibe Coding 难以胜任
- 上下文丢失：随着项目增长，AI 难以维持对整体架构和需求的理解

- **什么是 Spec Coding (Spec-Driven Development) ?**
 - 规范驱动开发：先写清楚 "要做什么" 和 "为什么", 再让 AI 生成 "怎么做"
 - 规范 (Spec) 成为可执行的核心产物, 而非传统开发中被丢弃的脚手架
 - 通过多步骤精炼流程, 而非一次性从 prompt 生成代码
- **Vibe Coding vs. Spec Coding**
 - Vibe Coding: "帮我做一个照片管理应用" -> 一次性生成 -> 不满意再改
 - Spec Coding: 需求规范 -> 技术方案 -> 任务拆解 -> 逐步实现 -> 自动验证
 - 核心区别: Spec Coding 强调意图先行、规范优先、多步精炼, **可以理解为一个软件工程 Skill**



- **什么是 GitHub Spec Kit?**

- GitHub 官方开源的 Spec-Driven Development 工具包
- 帮助开发者聚焦产品场景和可预测结果，而不是从零开始 Vibe Coding
- 已获得 8w+ GitHub Stars，支持几乎所有主流 AI 编程工具

- **核心组件**

- Specify CLI：命令行工具，用于初始化项目、检查环境
- Slash Commands：在 AI 编程工具中使用的一系列结构化命令（斜杠命令）
- 模板系统：规范模板、方案模板、任务模板等，确保输出一致性

- **安装方式**

- `uv tool install specify-cli --from git+https://github.com/github/spec-kit.git`



- **Step 1: 建立项目宪法 (Constitution)**
 - /speckit.constitution —— 创建项目的治理原则和开发规范
 - 定义代码质量、测试标准、用户体验一致性、性能要求等
 - 后续**所有开发阶段**都会参考这份 Constitution 作为决策依据
- **Step 2: 创建功能规范 (Spec)**
 - /speckit.specify —— 描述你要构建什么、为什么要构建
 - 关注 "做什么" 和 "为什么", **不需要指定技术栈**
 - 产出包含用户故事和功能需求的规范文档
- **Step 3: 需求澄清 (Clarify)**
 - /speckit.clarify —— 结构化地澄清规范中不明确的地方
 - 推荐在制定技术方案之前完成, 减少后续返工



- **Step 4: 制定技术方案 (Plan)**
 - /speckit.plan —— 在这一步指定技术栈和架构选型
 - 产出：实现方案、数据模型、API 规范、研究文档等
 - 可以让 AI 对快速变化的技术栈进行针对性调研
- **Step 5: 任务拆解 (Tasks)**
 - /speckit.tasks —— 从实现方案中自动生成可执行的任务列表
 - 按用户故事组织、管理依赖关系、**标记可并行任务**
 - 包含文件路径、测试驱动结构和检查点验证
- **Step 6: 执行实现 (Implement)**
 - /speckit.implement —— 按照任务列表逐步执行实现
 - 验证前置条件 -> 解析任务 -> 按序执行 -> 处理错误



- **项目启动时**
 - 跑一次 `/speckit.constitution`，建立项目开发规范
 - 此后除非原则变更，否则不再需要运行此命令
- **开发功能 A**（如前端页面）
 - 走完整 Spec 流程
 - `/speckit.specify -> /speckit.clarify -> /speckit.plan -> /speckit.tasks -> /speckit.implement`
- **微调功能 A**（改颜色、修 Bug 等）
 - 不需要使用 `/speckit` 命令，直接用自然语言（Vibe Coding）即可
- **开发功能 B**（如集成 RAG 系统）
 - 开启新一轮 Spec 循环（`specify -> ... -> implement`）
 - `/speckit.specify` 会自动创建新的 Git 分支



- **Spec Kit 支持几乎所有主流 AI 编程工具**
 - Claude Code、Cursor、Trae、GitHub Copilot、Codex、Gemini CLI、Kiro 等
- **初始化命令示例**
 - specify init my-project (可指定 AI 编程工具, 如 --ai claude)
 - specify init . (在当前目录初始化)
- **可选的质量保障命令**
 - /speckit.analyze —— 跨产物一致性分析 (在 /speckit.tasks 后使用)
 - /speckit.checklist —— 生成自定义质量检查清单, 验证需求完整性和一致性 (在 /speckit.plan 后使用)

- **InsForge 是什么?**
 - 专为 AI 辅助开发构建的后端平台 (Backend as a Service, BaaS)
 - 传统 BaaS (如 Supabase, Firebase) 面向人类开发者设计 (Dashboard + SDK) , 而 InsForge 原生支持 MCP / Skill, 使 Agent 能够直接理解并完成全部的后端操作, 是 **AI-Native 的 BaaS**
- **核心功能**
 - Authentication: 用户管理、身份认证和会话管理
 - Database: PostgreSQL (功能最全面的开源关系型数据库)
 - Storage: 对象存储 (提供 URL 访问链接)
 - AI Integration: GPT, Claude, Gemini, Grok, DeepSeek, GLM, Kimi, MiniMax
- **使用方式: 通过 MCP / Skill 连接 AI 编程工具, Agent 可直接调用后端能力**

- **Spec Coding 核心要点**

- Spec Coding 是一种结构化的 AI 辅助开发方法，通过先定义规范再实现代码来提升质量
- 与 Vibe Coding 的核心区别：多步精炼、规范优先、意图驱动
- GitHub Spec Kit 提供了完整的工具链支持，从规范创建到代码实现

- **Spec Kit workflow 回顾**

- Constitution (开发规范) -> Specify (功能规范) -> Clarify (需求澄清)
- -> Plan (技术方案) -> Tasks (任务拆解) -> Implement (执行实现)

- **Why InsForge?**

- 专为 AI 辅助开发设计的 BaaS 平台，通过 MCP / Skill 连接 Agent 与后端

- **在实验中的应用**

- 使用 Spec Kit 配合 AI 编程工具，基于 InsForge 后端和实验 1 的数据，完成全栈 Web 应用开发

